Enterprise Text-to-SQL API

48-Hour Build Challenge, NST

The 48-Hour Mission. Build a FastAPI microservice that converts natural language database questions into executable SQL. Your system must: (1) retrieve relevant schema tables using semantic search, (2) generate SQL via an LLM with retrieved context, (3) validate and execute queries, (4) measure performance on a benchmark dataset **(optional)**. Benchmarking is kept optional considering the time constraint.

DATASET TO WORK WITH & REFERENCES

Dataset: Take the Beaver dataset from HuggingFace (Beaver Dataset Link, Click here). **You have to develop your system using this dataset only.**

Paper: Refer to this paper to deeply understand the dataset and the problem you are solving in this challenge (Beaver Paper Link, Click here).

FastAPI: If you are new to FastAPI, refer to the official documentation to get started . FastAPI Page Link, Click here. **You must only use this to develop the API and its endpoints.**

WHAT YOU ARE BUILDING

An enterprise user asks: *"Show me departments ranked by total enrollment, excluding online courses."* Your system must:

1. **Retrieve tables**: Identify that the query needs `departments`, `enrollments`, and `courses` tables from a schema with 40+ tables.
2. **Understand context**: Map "enrollment" → `enrollments.student_count`, infer that "online courses" is stored as a course attribute.
3. **Generate SQL**: Use an LLM with the retrieved schema to produce valid, optimised SQL.
4. **Validate and execute**: Verify syntax, execute on a test database, return results.
5. **Measure accuracy**: Report retrieval precision, SQL correctness, and end-to-end success rate.

This mirrors production text-to-SQL systems used in enterprise BI tools (e.g., Databricks Assistant, Mode Analytics Query Engine, etc.).

SYSTEM ARCHITECTURE

A high-level overview of what you will build:

```
User Question
    ↓
Schema Context Builder
    ↓
LLM Layer
    ↓
Validation Layer
    ↓
DB Layer
    ↓
Metrics Layer
    ↓
FastAPI Response (SQL + metrics)
```

REQUIRED API ENDPOINTS

Your FastAPI service must expose these three core endpoints:

POST /retrieve

Retrieve relevant tables for a user question.

Listing 1: /retrieve Request

```
{
  question: Which departments have more than 100 students?
}
```

Listing 2: /retrieve Response (200 OK)

```
{
  retrieved_tables: [departments, enrollments],
  scores: [0.92, 0.87],
  confidence: 0.90,
  details: {
    departments: {
      relevance_score: 0.92,
      reason: Question asks about departments directly
    },
    enrollments: {
      relevance_score: 0.87,
      reason: Needed to count students per department
    }
  }
}
```

POST /generate-sql

Generate SQL from a question using the retrieval + LLM pipeline.

Listing 3: /generate-sql Request

```
{
  question: Which departments have more than 100 students?,
  use_retrieved_context: true
}
```

Listing 4: /generate-sql Response (200 OK)

```
{
```

```
  sql: SELECT d.dept_name, COUNT(e.student_id) as total_students FROM departments d JOIN
      enrollments e ON d.dept_id = e.dept_id GROUP BY d.dept_name HAVING COUNT(e.
      student_id) > 100,
  retrieved_tables: [departments, enrollments],
  is_valid_syntax: true,
  parsing_errors: null,
  confidence: 0.85,
  prompt_used: ...full prompt sent to LLM...
}
```

POST /benchmark

Evaluate performance on a benchmark dataset of 20–30 questions.

Listing 5: /benchmark Response (200 OK)

```
{
  total_queries: 25,
  metrics: {
    retrieval_recall_at_5: 0.88,
    retrieval_recall_at_10: 0.92,
    sql_exact_match_accuracy: 0.52,
    sql_execution_match_accuracy: 0.68,
    parsing_success_rate: 0.96,
    average_latency_ms: 850
  },
  subtask_breakdown: {
    multi_table_retrieval: 0.88,
    column_mapping: 0.72,
    join_detection: 0.65,
    domain_knowledge: 0.58
  },
  error_analysis: {
    retrieval_failures: 3,
    parsing_failures: 1,
    execution_failures: 8,
    logic_errors: 6
  }
}
```

TECHNICAL REQUIREMENTS

LLM Integration

- Use LLM of your choice. You can refer to HuggingFace for free access to open-source models.
- Construct a prompt that includes: user question, retrieved table schemas, instructions, in-context examples.
- Handle LLM API errors gracefully (retries, fallback).
- Log the full prompt and LLM response for debugging.

FastAPI Implementation

Must Implement:

- Pydantic request/response models for all endpoints.
- Proper HTTP status codes (200, 400, 500).
- Input validation (non-empty question, reasonable length limits).

- Auto-generated OpenAPI documentation (`/docs`).
- Structured JSON logging.
- Modular code: separate files for retrieval, LLM, validation, execution.

SUBMISSION CHECKLIST

By the end of 48 hours, submit:

- **GitHub Repository link** with:
  - All source code.
  - Readme.md file explaining your system architecture, components, approach, end-points, etc.
  - Screenshots of your FastAPI service running on localhost and responses for various endpoints (`/retrieve`, `/generate-sql`, `/benchmark`) demonstrating the functioning of your service.

Send your final GitHub repository link to `rajpreet.singh@tum.de` by June 3 2026, 11:59 pm IST

TIPS & BEST PRACTICES

Time Management: Don't get stuck on one piece. Aim for a working end-to-end system by hour 24. Refinement and optimization come after.

Retrieval First: Retrieval quality heavily influences SQL generation. Invest time here. A good retrieval system (85%+ recall@5) can carry weak LLM prompts.

Database Setup: Use SQLite for simplicity during development. Migrate to PostgreSQL if needed, but don't waste time on DB infrastructure.

Error Handling: Log everything. If a query fails, you need to know: did retrieval fail? LLM? Validation? Execution? Detailed logs save debugging time.

In-Context Examples: Use 3–5 real benchmark examples showing correct SQL for similar questions. This is one of the highest-leverage optimizations.

Final Thoughts. This challenge mirrors real-world text-to-SQL systems in production. You will encounter the same hard problems: ambiguous questions, complex schemas, LLM hallucinations, and edge cases. By shipping a working system in 48 hours, you will gain deep intuition about the trade-offs: speed vs. accuracy, coverage vs. precision, and how to debug systems that depend on black-box LLMs. Good luck! and be Creative !